

Tydzień 6; grupa średniozaawansowana

01.04.2025 (to nie żart, zadania należy wykonać)

Zadanie pierwsze_tablice

Stwórz dwie tablice o długości 20. Pierwszą z nich wypełnij kolejnymi potęgami liczby π . Następnie, korzystając z pierwszej tablicy, wypełnij drugą tablicę tymi samymi liczbami w odwróconej kolejności. Spróbuj napisać ten program bez używania funkcji `pow`.

Zadanie Tablice_wskaźniki

Stwórz kolejną wersję funkcji, która zamienia ze sobą dwie liczby lub dwa znaki, a jej argumentami są tym razem dwa wskaźniki. Sprawdź, czy liczby lub znaki zostały zamienione ($\rightarrow$ materiały wykładowe).

Zadanie expTylor

Napisz funkcję `expTaylor`, która dla zadanych argumentów `double x` i `int kmax` zwraca przybliżenie wartości funkcji wykładniczej za pomocą szeregu Taylora:

$$e^x \approx \sum_{n=0}^{kmax} \frac{x^n}{n!} \quad (1)$$

Obliczanie wartości $n!$ umieść w osobnej funkcji.

W funkcji `main()`:

- Poproś użytkownika o podanie wartości x oraz k_{max} .
- Wypisz przybliżoną wartość funkcji e^x obliczoną za pomocą funkcji `expTaylor`.
- Porównaj wynik z wartością zwracaną przez funkcję `exp(x)` z biblioteki `<cmath>`.
- Wypisz różnicę pomiędzy przybliżeniem a rzeczywistą wartością.

Na koniec zastanów się, czy konieczne jest każdorazowe obliczanie rosnących potęg i silni. Podziel wyraz a_{n+1} przez a_n , wyciągnij wnioski i zmodyfikuj funkcję `expTaylor` w taki sposób, aby eliminować zbędne obliczenia.

Zadanie wektory

Napisz program, który wypełnia wektor o długości n całkowitymi liczbami losowymi z przedziału od 0 do 1000 i znajdzie największą z nich.

Liczba n powinna być przekazywana do wnętrza programu jako argument wywołania programu. Zabezpiecz program przed uruchomieniem go bez podania wartości n .

Przykłady wywołania programu:

- Dla $n = 20$: `./a.out 20`
- Alternatywnie: `./a.out -n 20`

Podział programu na funkcje

Program powinien być podzielony na następujące funkcje:

- Funkcja do znajdowania indeksu elementu wektora, w którym znajduje się największa liczba (bez użycia standardowych algorytmów).
- Funkcja do wypełniania wektora wartościami losowymi.
- Funkcja do wypisywania zawartości wektora na ekran.
- Funkcja do wczytywania argumentów wywołania programu (tylko w wersji zaawansowanej).

Użycie funkcji losujących

Program powinien używać funkcji `rand()` z biblioteki `cstdlib`, aby generować liczby losowe w zakresie 0–1000:

```
rand() % 1001;
```

Aby uzyskać różne liczby losowe przy każdym uruchomieniu, należy na początku programu ustawić:

```
srand(time(0));
```

Porównanie z algorytmem standardowym

Porównaj wynik swojego algorytmu z wynikiem działania standardowego algorytmu `max_element` z biblioteki `algorithm`:

```
cout << *max_element(vec.begin(), vec.end()) << endl;
```

****Zadanie Srebrny_glob**

W 1903 r. Jerzy Żulawski opublikował książkę "Na Srebrnym Globie: rękopis z książyca." Twoim zadaniem jest stworzenie programu, który pomoże w analizie rękopisu. Program powinien:

- Wczytać od użytkownika liczbę n , oznaczającą ilość zdań zapisanych w rękopisie.
- Dynamicznie przydzielić pamięć na tablicę wskaźników do zdań.
- Wczytać n zdań od użytkownika i zapisać je w tablicy. Można je skopiować z rękopisu (należy pobrać plik, aby link działał).
- Odkryć tajemniczą anomalię i zamienić miejscami pierwsze oraz ostatnie zdanie przy użyciu wskaźników.
- Odszyfrować ukryte znaczenie, znajdując najdłuższe i najkrótsze zdanie.
- Przenieść zdania do obiektu klasy `std::vector<std::string>` i posortować je według długości, aby zrozumieć hierarchię informacji.
- Odwrócić kolejność zdań w wektorze, aby odkryć ukryty przekaz.
- Wyświetlić zawartość przetworzonych zdań i zaprezentować analizę.
- Zwolnić dynamicznie przydzieloną pamięć, aby nie pozostawić śladów po analizie.